



Module 3

Team Members:

Mary Burns, Ryan Sloan

Project Title:

Investigating Apoptosis Gene Pathways in Breast Cancer using Metadata Disease Background:

Project Goal:

To determine whether the expression of apoptosis-related genes (such as TP53, BCL2, BAX, and CASP3) can be used to predict differences in patient survival outcomes among individuals diagnosed with breast cancer (BRCA) using data from The Cancer Genome Atlas (TCGA).

Research Question

Can apoptosis gene expression patterns predict whether a breast cancer patient has a short (<3 years) or long (≥ 3 years) survival outcome?

Disease Background:

- Cancer hallmark focus: Apoptosis
- Overview of hallmark: Apoptosis is the normal, planned death of damaged or old cells. It's a natural process that keeps the body healthy by removing cells that could become harmful if they stay alive. When apoptosis happens correctly, it prevents damaged DNA from being copied and forming cancer. But when apoptosis is disrupted or fails to occur, those damaged cells survive and continue dividing, which can lead to tumor growth.
- Genes associated with hallmark to be studied (describe the role of each gene, signaling pathway, or gene set you are going to investigate):
TP53 (p53): A major tumor suppressor that detects genomic damage and can induce apoptosis. When mutated, which occurs in over half of

human cancers, this pathway fails to activate cell death in response to stress. BCL-2 family: A group of genes that regulate the intrinsic mitochondrial pathway of apoptosis. Anti-apoptotic members like BCL-2 and BCL-XL prevent the release of pro-death signals, while pro-apoptotic members like BAX and BAK promote cell death. Caspases: The executioner proteases that carry out the apoptotic process by degrading key structural and regulatory proteins. PI3K-AKT pathway: A signaling cascade that suppresses apoptosis and supports survival. Overactivation of this pathway in many tumors contributes to resistance to cell death. <https://my.clevelandclinic.org/health/articles/apoptosis>

Cancer Type Focus: Breast Cancer

- Prevalence & incidence: Prevalence: a measure used in epidemiology and public health to measure the percentage of the population affected by a disease $\text{Prevalence} = (\# \text{ of existing cases}) / (\text{population}) * 100$ Breast cancer is both the second most common cancer and the second leading cause of cancer death among women. Research shows that approximately one in eight women in the United States will receive a breast cancer diagnosis in their lifetime. <https://www.nationalbreastcancer.org/breast-cancer-facts/> Incidence: number of new cases of a disease $\text{Incidence} = (\# \text{ of new cases}) / (\text{population})$ As of September 2025, approximately 316,000 women and 3,000 men have been diagnosed with breast cancer this year alone, and an additional 50,000 are expected to receive a diagnosis. <https://www.cdc.gov/breast-cancer/statistics/index.html>
- Risk factors (genetic, lifestyle) & Societal determinants Research shows that the two most significant factors in the development of breast cancer are increasing age (>50) and being a woman. Genetic risk factors for this type of cancer include early onset of periods, late onset of menopause, having dense breasts, personal and/or familial history of breast and ovarian cancers, and prior usage of radiation therapies. Lifestyle risk factors include living a sedentary lifestyle, obesity, pregnancy after age 30, never breastfeeding, undergoing hormone replacement therapies (especially for estrogen and progesterone), and heavy consumption of alcohol. Societally, breast cancer develops fairly consistently across all demographics, but groups who are unable to access/afford treatment have increased chances of worsening symptoms and death. <https://www.cdc.gov/breast-cancer/risk-factors/>

[index.html https://www.ajmc.com/view/racial-ethnic-and-socioeconomic-gaps-persist-in-us-female-specific-cancer-burden#:~:text=Persistent%20racial%2C%20ethnic%2C%20and%20socioeconomic%20cancers%20in%20the%20US](https://www.ajmc.com/view/racial-ethnic-and-socioeconomic-gaps-persist-in-us-female-specific-cancer-burden#:~:text=Persistent%20racial%2C%20ethnic%2C%20and%20socioeconomic%20cancers%20in%20the%20US).

- Standard of care treatments (& reimbursement): standard of treatment
The standard treatments for breast cancer vary depending on the patient but generally include both local and systemic options. Local treatments, such as surgery and radiation, focus on removing or destroying the cancer in a specific area. Systemic treatments, which can be given by injection, orally, or directly into the bloodstream, include chemotherapy, hormone therapy, targeted drugs, immunotherapy, and biological therapy. There are also many programs available to help with the financial and practical challenges that come with treatment. Reimbursement and support options include Supplemental Security Income (SSI), Social Security Disability Insurance (SSDI), and Temporary Assistance for Needy Families (TANF), which assist patients who are unable to work. For housing support, programs like the American Cancer Society Hope Lodge, Extended Stay America, the Healthcare Hospitality Network, and the Ronald McDonald House offer places for patients and families to stay. Transportation assistance is available through programs such as the American Cancer Society Road to Recovery, Blood Cancer United, Medicare, and Medicaid. For help with food costs, patients can turn to Meals on Wheels or the Supplemental Nutrition Assistance Program (SNAP). Even beyond these, there are many other resources in place to support patients undergoing breast cancer treatment. Treatment Options for Breast Cancer American Cancer Society <https://www.cancer.org> > ... > Breast Cancer <https://www.cdc.gov/breast-cancer/treatment/index.html> <https://www.cancer.org/cancer/types/breast-cancer/treatment.html> <https://www.cancer.org/cancer/financial-insurance-matters/managing-health-insurance/programs-and-resources-to-help-with-cancer-related-expenses.html>
- Biological mechanisms (anatomy, organ physiology, cell & molecular physiology) Breast cancer is a molecularly diverse disease that begins in the ductal and lobular epithelial cells of the breast. At the organ level, changes in hormone signaling, such as estrogen and progesterone, and altered growth factor activity can drive abnormal cell growth. On the cellular and molecular level, breast cancers are grouped

by receptor status, which includes hormone receptor-positive, HER2-positive, and triple-negative types that reflect different genetic mutations and signaling behaviors. Key pathways such as RAS, RAF, MEK, ERK, and PI3K, AKT, and mTOR promote continuous cell growth and survival, while the loss of tumor suppressor genes such as TP53 and RB1 removes normal cell cycle control. Many breast cancers also avoid apoptosis through increased BCL2 expression or reduced caspase activity, allowing damaged cells to survive. In some cases, the loss of adhesion proteins such as E-cadherin (also known as CDH1) weakens tissue structure and supports invasion and metastasis.

https://www.mdpi.com/journal/cells/special_issues/molecular_cellular_breast_cancer <https://emedicine.medscape.com/article/1947145-overview?form=fpf#a5> <https://hms.harvard.edu/news/how-breast-cancer-arises> <https://ar.iiarjournals.org/content/38/5/2607>

Methods:

We used logistic regression, a supervised learning method that predicts whether a sample is tumor or normal based on gene expression. The model finds patterns in the data by adjusting how much weight it gives each gene. It measures how well it's doing using log loss, and stops training once it can't improve accuracy any further.

Data-Set:

The data from this project comes from two papers building upon The Cancer Gene Atlas [TCGA], which synthesizes data from various tumors, providing a comprehensive database of information about the cancer and the patient. These papers are the Pan-Cancer Clinical Data Resource (Liu et al., 2018) and the Reprocessed TCGA RNA-Sequencing Dataset (Rahman et al., 2015). The first paper, Pan-Cancer Clinical Data Resource, is a paper breaking down around 11,000 cases of 33 different types of cancers, mentioning cancer progression, patient demographics, and survival. The Reprocessed TCGA RNA-Sequencing Dataset used the Rsubread Pipeline to process gene-expression data from 9,264 tumor and 741 normal samples. Of these 10,005 data points, 23 cancer types are represented. The data created from the Rsubread Pipeline created integer values in units of FPKM and TPM.

```
In [97]: import pandas as pd
```

```

# Load the metadata CSV
metadata = pd.read_csv("GSE62944_metadata.csv")

# Filter rows where cancer_type is BRCA
brca_metadata = metadata.loc[metadata['cancer_type'] == 'BRCA']

# When time comes, at this point, we will select which pieces of the metadata
# important to us after further development of our question

# Save the filtered metadata to a new CSV
brca_metadata.to_csv("GSE62944_metadata_BRCA.csv", index=False)

# (Code assisted by Copilot)

```

```

In [98]: import pandas as pd

# 1. Load BRCA metadata (tumor only like before)
metadata_brca = pd.read_csv("GSE62944_metadata_BRCA.csv")
metadata_brca["sample_clean"] = metadata_brca["sample"].str.replace(".", "-",

def sample_type_from_id(sample_id):
    if pd.isna(sample_id):
        return "Unknown"
    parts = str(sample_id).split('-')
    if len(parts) >= 4:
        code = parts[3][:2]
        if code in ["01", "03"]:
            return "Primary Tumor"
        elif code == "11":
            return "Solid Tissue Normal"
        else:
            return "Other"
    else:
        return "Unknown"

metadata_brca["sample_type"] = metadata_brca["sample_clean"].apply(sample_type_from_id)
metadata_brca_tumor = metadata_brca[metadata_brca["sample_type"] == "Primary Tumor"]

# 2. Load survival and rename + truncate barcode
surv = pd.read_csv("subsamped_TCGA_CDR_survival.csv")
surv = surv.rename(columns={"Unnamed: 0": "bcr_patient_barcode"})

metadata_brca_tumor["bcr_patient_barcode"] = metadata_brca_tumor["bcr_patient_barcode"]
surv["bcr_patient_barcode"] = surv["bcr_patient_barcode"].astype(str).str[:12]

# 3. Merge and see how many have survival
merged_test = metadata_brca_tumor.merge(surv, on="bcr_patient_barcode", how="inner")

print("Number of BRCA tumor samples with any survival row:", merged_test.shape[0])

```

```
# How many have OS.time not missing?
merged_test_non_missing = merged_test.dropna(subset=["OS.time"])
print("Number of BRCA tumor samples with non-missing OS.time:", merged_test_non_missing["OS.time"].describe())
```

```
Number of BRCA tumor samples with any survival row: 76
Number of BRCA tumor samples with non-missing OS.time: 75
count      75.000000
mean       1398.120000
std        1700.116951
min         0.000000
25%        444.000000
50%        767.000000
75%       1690.500000
max       8605.000000
Name: OS.time, dtype: float64
```

This code will import all the libraries we need and set up basic plot styling.

```
In [99]: import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_auc_score,
    roc_curve
)

import seaborn as sns
import matplotlib.pyplot as plt

sns.set(style="whitegrid")
plt.rcParams["figure.figsize"] = (6, 4)
```

This code will load the BRCA metadata, clean sample IDs, and keep only primary tumor samples.

```
In [100]: metadata_brca = pd.read_csv("GSE62944_metadata_BRCA.csv")

# Clean sample IDs (TCGA uses '-' not '.')
metadata_brca["sample_clean"] = metadata_brca["sample"].str.replace(".", "-")

# Helper function to decode TCGA sample type from barcode
def sample_type_from_id(sample_id):
    if pd.isna(sample_id):
```

```

        return "Unknown"
    parts = str(sample_id).split('-')
    if len(parts) >= 4:
        code = parts[3][:2]
        if code in ["01", "03"]:
            return "Primary Tumor"
        elif code == "11":
            return "Solid Tissue Normal"
        else:
            return "Other"
    return "Unknown"

metadata_brca["sample_type"] = metadata_brca["sample_clean"].apply(sample_type)

# Keep only primary tumor samples
metadata_brca_tumor = metadata_brca[metadata_brca["sample_type"] == "Primary Tumor"]
print("Number of BRCA tumor samples:", metadata_brca_tumor.shape[0])

```

Number of BRCA tumor samples: 80

This code will load the survival data, align patient barcodes, merge with tumor metadata, and create a short vs long survival label based on a 4-year cutoff.

```

In [101]: # Load survival data and rename ID column
surv = pd.read_csv("subsamped_TCGA_CDR_survival.csv")
surv = surv.rename(columns={"Unnamed: 0": "bcr_patient_barcode"})

# Derive patient barcode (first 12 chars) from sample_clean
metadata_brca_tumor["bcr_patient_barcode"] = metadata_brca_tumor["sample_clean"]
surv["bcr_patient_barcode"] = surv["bcr_patient_barcode"].astype(str).str[:12]

# Merge metadata + survival
merged = metadata_brca_tumor.merge(surv, on="bcr_patient_barcode", how="inner")

# Drop samples with missing survival time
merged = merged.dropna(subset=["OS.time"])
print("Number of BRCA tumor samples with OS.time:", merged.shape[0])

# Define short survival (< 4 years) = 1, long survival (>= 4 years) = 0
threshold_days = 4 * 365 # 1460 days
merged["short_survival"] = (merged["OS.time"] < threshold_days).astype(int)

print("Short vs Long survival counts (4-year threshold):")
print(merged["short_survival"].value_counts())

```

Number of BRCA tumor samples with OS.time: 75

Short vs Long survival counts (4-year threshold):

short_survival

1 53

0 22

Name: count, dtype: int64

This code will load the expression matrix, match samples with survival data, select

apoptosis-related genes, and build the feature matrix X and label vector y.

```
In [102... # Load expression data (log2 TPM)
expr = pd.read_csv("GSE62944_subsample_log2TPM.csv", index_col=0)
expr.columns = expr.columns.str.replace(".", "-", regex=False)

# Match expression columns to samples with survival data
final_samples = merged["sample_clean"].tolist()
expr_brca = expr[final_samples]

# Apoptosis-related genes of interest
genes_of_interest = ["TP53", "BCL2", "BAX", "CASP3"]

# Check if these gene names appear in the index
for g in genes_of_interest:
    if g not in expr_brca.index:
        hits = [x for x in expr_brca.index if g in x]
        print(f"{g} possible matches:", hits[:3])

# If needed, manually adjust to "GENE|ID" forms based on printed hits
# Example:
# genes_of_interest = ["TP53|7157", "BCL2|596", "BAX|581", "CASP3|836"]

# Build feature matrix (samples x genes)
X = expr_brca.loc[genes_of_interest].T
X = X.astype(float)

# Build label vector: short_survival (1) vs long_survival (0)
y = merged.set_index("sample_clean").loc[final_samples, "short_survival"]

print("Feature matrix shape:", X.shape)
print("Label distribution:\n", y.value_counts())
```

Feature matrix shape: (75, 4)

Label distribution:

short_survival

1 53

0 22

Name: count, dtype: int64

This code will run PCA on the standardized gene expression data and plot the first two principal components colored by survival group.

```
In [103... # Standardize features for PCA
scaler_full = StandardScaler()
X_scaled_full = scaler_full.fit_transform(X)

# Fit PCA with 2 components
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled_full)

# Build DataFrame for plotting
pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'], index=X.index)
```



```

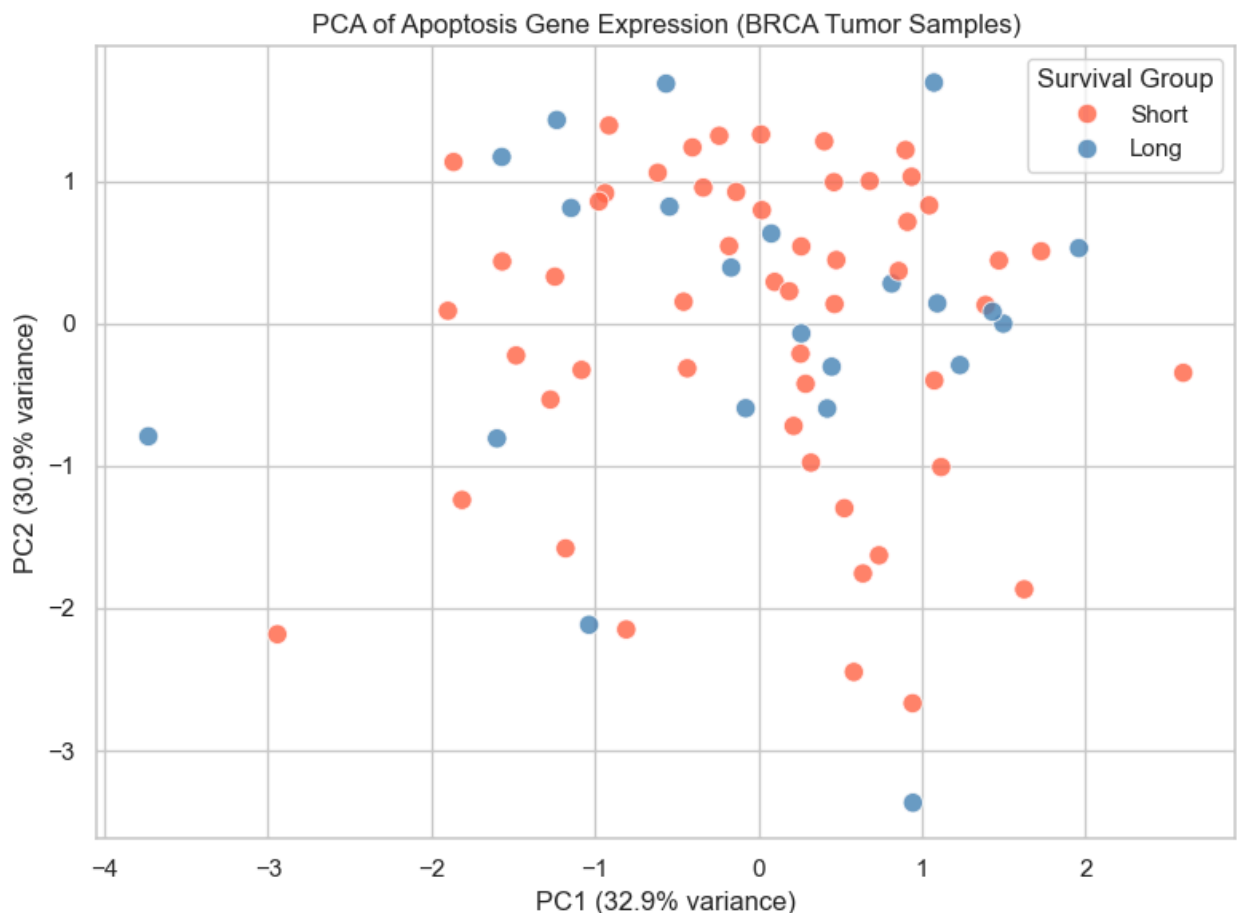
pca_df['Survival_Group'] = y.map({0: 'Long', 1: 'Short'})

print("Explained variance by components:", pca.explained_variance_ratio_)

# Plot PCA colored by survival group
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=pca_df, x='PC1', y='PC2', hue='Survival_Group',
    palette={'Long': 'steelblue', 'Short': 'tomato'},
    s=80, alpha=0.8
)
plt.title('PCA of Apoptosis Gene Expression (BRCA Tumor Samples)')
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)")
plt.legend(title='Survival Group')
plt.tight_layout()
plt.show()

```

Explained variance by components: [0.32931522 0.30945085]



This code will split the data into training and test sets and standardize the features for modeling.

```

In [104... X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

```

```

print("Train samples:", X_train.shape[0])
print("Test samples:", X_test.shape[0])

# Standardize on training data, apply same transform to test
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

Train samples: 52

Test samples: 23

This code will train a class-weighted logistic regression model on the scaled training data and compute predictions and probabilities.

```

In [105... model = LogisticRegression(max_iter=1000, class_weight="balanced")
model.fit(X_train_scaled, y_train)

# Predictions (class) and probabilities
y_train_pred = model.predict(X_train_scaled)
y_test_pred = model.predict(X_test_scaled)

y_train_proba = model.predict_proba(X_train_scaled)[:, 1]
y_test_proba = model.predict_proba(X_test_scaled)[:, 1]

print("Model training complete.")

```

Model training complete.

This code will print accuracy and a full classification report (precision, recall, F1) for both the training and test sets.

```

In [106... print("TRAIN SET SCORES")
print("Accuracy:", round(accuracy_score(y_train, y_train_pred), 3))
print(classification_report(y_train, y_train_pred))

print("\nTEST SET SCORES")
print("Accuracy:", round(accuracy_score(y_test, y_test_pred), 3))
print(classification_report(y_test, y_test_pred))

```

TRAIN SET SCORES

Accuracy: 0.558

	precision	recall	f1-score	support
0	0.35	0.60	0.44	15
1	0.77	0.54	0.63	37
accuracy			0.56	52
macro avg	0.56	0.57	0.54	52
weighted avg	0.65	0.56	0.58	52

TEST SET SCORES

Accuracy: 0.565

	precision	recall	f1-score	support
0	0.29	0.29	0.29	7
1	0.69	0.69	0.69	16
accuracy			0.57	23
macro avg	0.49	0.49	0.49	23
weighted avg	0.57	0.57	0.57	23

This code will create confusion matrix heatmaps for the training and test sets to show how often the model is right or wrong for each class.

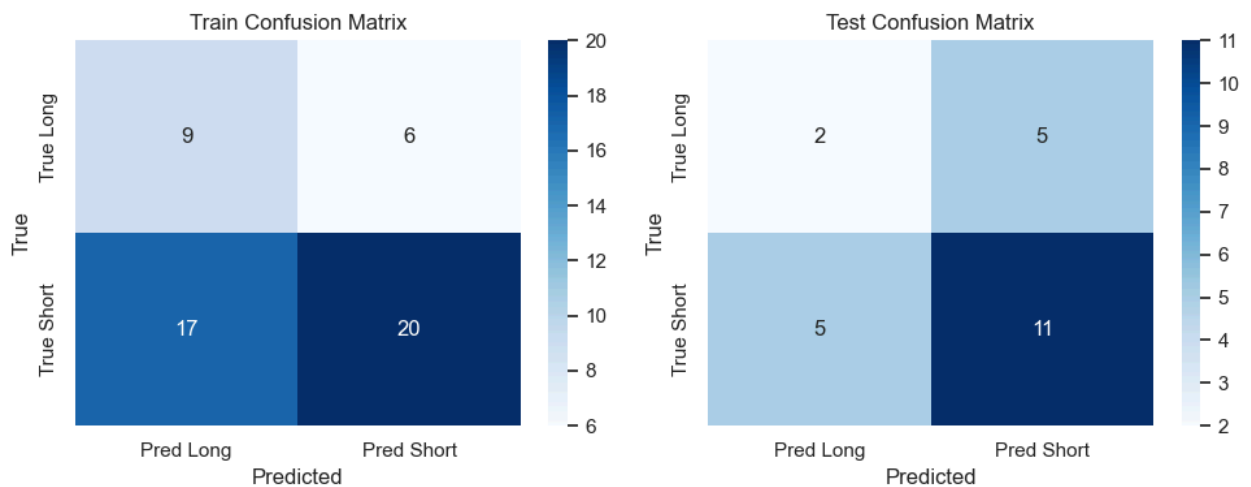
```
In [107... cm_train = confusion_matrix(y_train, y_train_pred)
cm_test = confusion_matrix(y_test, y_test_pred)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

sns.heatmap(
    cm_train, annot=True, fmt="d", cmap="Blues", ax=axes[0],
    xticklabels=["Pred Long", "Pred Short"],
    yticklabels=["True Long", "True Short"]
)
axes[0].set_title("Train Confusion Matrix")
axes[0].set_xlabel("Predicted")
axes[0].set_ylabel("True")

sns.heatmap(
    cm_test, annot=True, fmt="d", cmap="Blues", ax=axes[1],
    xticklabels=["Pred Long", "Pred Short"],
    yticklabels=["True Long", "True Short"]
)
axes[1].set_title("Test Confusion Matrix")
axes[1].set_xlabel("Predicted")
axes[1].set_ylabel("True")

plt.tight_layout()
plt.show()
```



This code will compute AUC for train and test and plot the ROC curve for the test set.

```
In [108... train_auc = roc_auc_score(y_train, y_train_proba)
test_auc = roc_auc_score(y_test, y_test_proba)

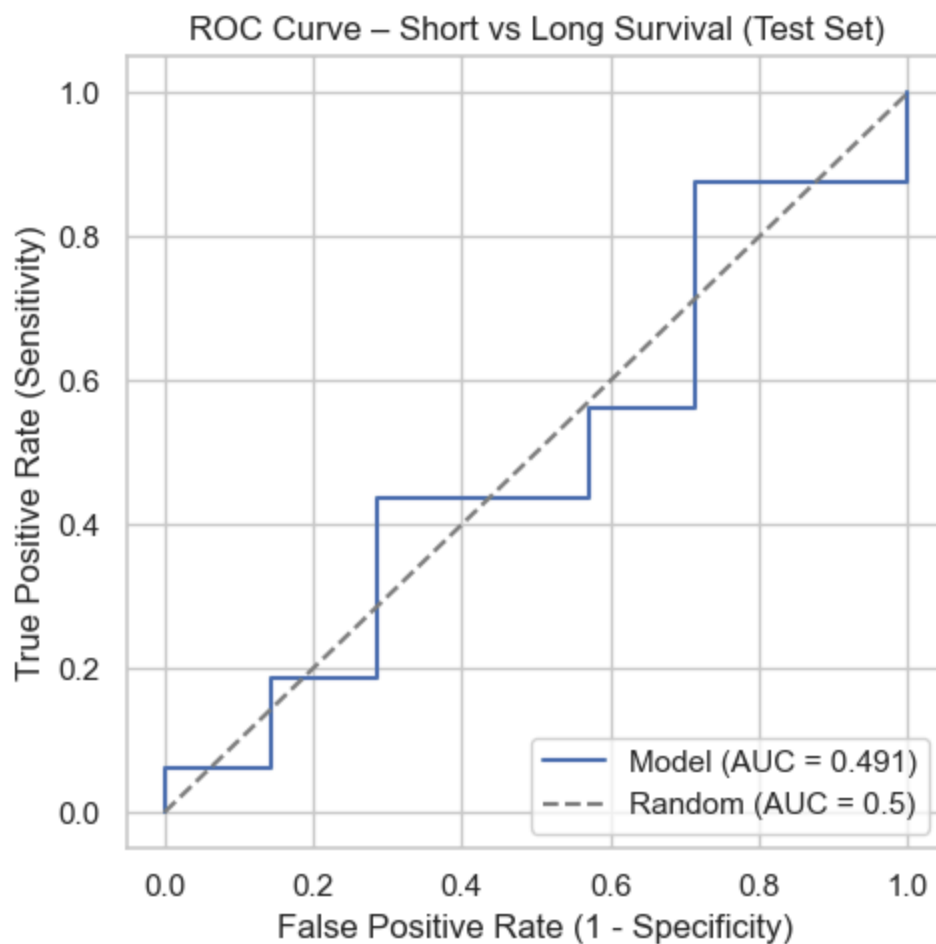
print("Train AUC:", round(train_auc, 3))
print("Test AUC:", round(test_auc, 3))

fpr, tpr, thresholds = roc_curve(y_test, y_test_proba)

plt.figure(figsize=(5, 5))
plt.plot(fpr, tpr, label=f"Model (AUC = {test_auc:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Random (AUC = 0.5)")
plt.xlabel("False Positive Rate (1 - Specificity)")
plt.ylabel("True Positive Rate (Sensitivity)")
plt.title("ROC Curve – Short vs Long Survival (Test Set)")
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

Train AUC: 0.672

Test AUC: 0.491



This code will test different decision thresholds (0.3, 0.5, 0.7) and show how accuracy, precision, recall, and F1 change on the test set.

```
In [109... def evaluate_threshold(threshold):
    y_test_custom = (y_test_proba >= threshold).astype(int)
    print(f"\n=== Threshold = {threshold} ===")
    print("Accuracy:", round(accuracy_score(y_test, y_test_custom), 3))
    print(classification_report(y_test, y_test_custom))

    for thr in [0.3, 0.5, 0.7]:
        evaluate_threshold(thr)
```

=== Threshold = 0.3 ===

Accuracy: 0.652

	precision	recall	f1-score	support
0	0.00	0.00	0.00	7
1	0.68	0.94	0.79	16
accuracy			0.65	23
macro avg	0.34	0.47	0.39	23
weighted avg	0.47	0.65	0.55	23

=== Threshold = 0.5 ===

Accuracy: 0.565

	precision	recall	f1-score	support
0	0.29	0.29	0.29	7
1	0.69	0.69	0.69	16
accuracy			0.57	23
macro avg	0.49	0.49	0.49	23
weighted avg	0.57	0.57	0.57	23

=== Threshold = 0.7 ===

Accuracy: 0.478

	precision	recall	f1-score	support
0	0.33	0.71	0.45	7
1	0.75	0.38	0.50	16
accuracy			0.48	23
macro avg	0.54	0.54	0.48	23
weighted avg	0.62	0.48	0.49	23

This code will compute McFadden's pseudo- R^2 to summarize overall model fit for the logistic regression.

```
In [110... eps = 1e-15 # to avoid log(0)

# Log-likelihood of the fitted model
ll_full = np.sum(
    y_test * np.log(y_test_proba + eps) +
    (1 - y_test) * np.log(1 - y_test_proba + eps)
)

# Log-likelihood of a null model that always predicts mean(y)
p_null = np.mean(y_test)
ll_null = np.sum(
    y_test * np.log(p_null + eps) +
    (1 - y_test) * np.log(1 - p_null + eps)
)
```

```
r2_mcfadden = 1 - (ll_full / ll_null)
print(f"\nMcFadden's pseudo-R2: {r2_mcfadden:.4f}")
```

McFadden's pseudo-R²: -0.1581

This code will print the logistic regression coefficients and plot them so you can see which genes push toward short vs long survival.

```
In [111]: coef_df = pd.DataFrame({
            "Gene": X.columns,
            "Coefficient": model.coef_[0]
        }).sort_values("Coefficient", ascending=False)

print("\nLogistic Regression Coefficients:")
print(coef_df)

plt.figure(figsize=(6, 4))
sns.barplot(data=coef_df, x="Coefficient", y="Gene", palette="coolwarm")
plt.axvline(0, color="black", linewidth=1)
plt.title("Gene Expression Effect on Short Survival (4-Year Threshold)")
plt.xlabel("Logistic Regression Coefficient")
plt.ylabel("Gene")
plt.tight_layout()
plt.show()
```

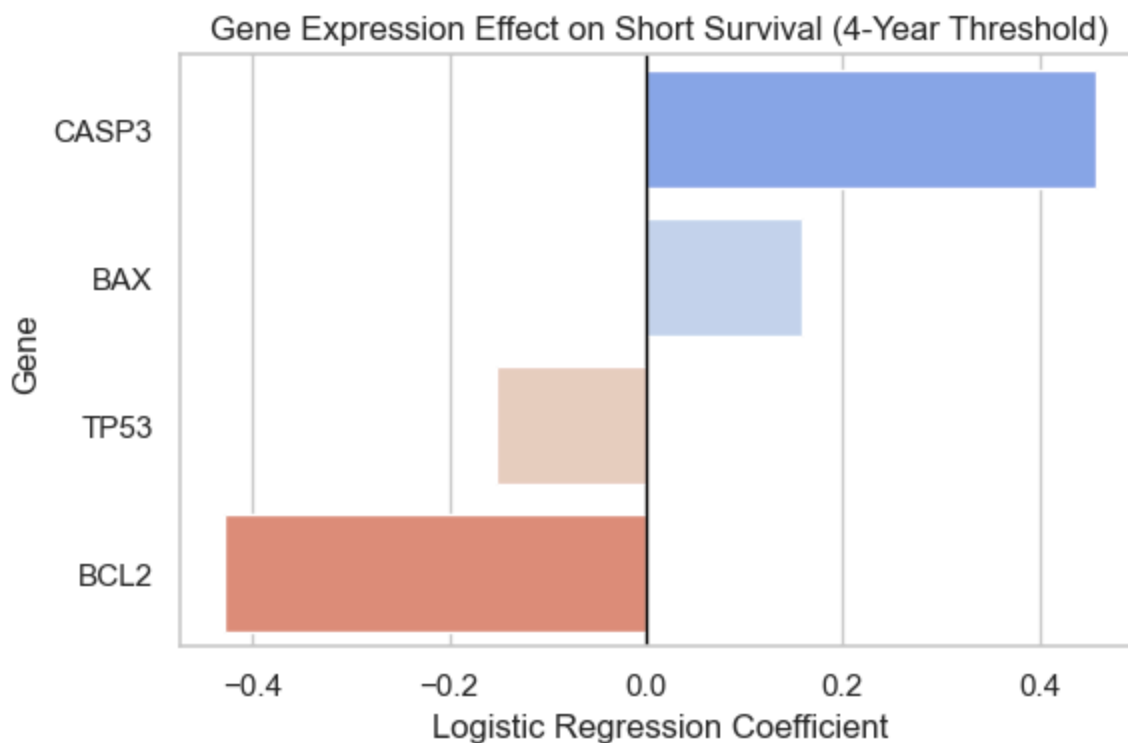
Logistic Regression Coefficients:

	Gene	Coefficient
3	CASP3	0.457807
2	BAX	0.158872
0	TP53	-0.151765
1	BCL2	-0.429741

```
/var/folders/3j/ht3ssvz15lgcklwx546s3r6m0000gn/T/ipykernel_54374/3169342717.p
y:10: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=coef_df, x="Coefficient", y="Gene", palette="coolwarm")
```



Verify and validate your analysis:

To analyze the effectiveness of our model, we used the ROC (Receiver Operating Characteristics) curve, as well as its associated AUC (Area Under the Curve). This method compares the true positive rate and the false positive rate across our different specifications (short-term and long-term). The AUC builds off of this by providing numerical data to show the model's accuracy, providing a value within a range of 0-1, with 1 being perfect distinction between categories. Using train-test data, our model yielded a AUC of 0.491, which is comparable to random guessing (AUC = 0.5). From this, we can see that our model is not very accurate in terms of classifying data accurately. This result is shown further in our confusion matrices above, which consistently was unable to correctly distinguish long-term survival data. Furthermore, our negative value for the McFadden's pseudo R^2 echoes these findings, showing our fit of data to be poor. To further better our model, more raw data could be used to refine mechanisms. Considering all factors above, the model we devised underfits the data. When looking at literature about our data, the gene shown to be most correlated from our analysis, CASP-3, external literature showed that the gene has a confusing relationship with cells, both helping with and resisting apoptosis depending on the context of the cell, providing some insight why the logistic regression coefficient was only 0.46.

https://pmc.ncbi.nlm.nih.gov/articles/PMC9106709/pdf/JCB_202201159.pdf

Conclusions and Ethical Implications:

Due to our model being underfitted and not the most accurate, no definitive conclusion can be drawn from our data set. However, from our model, key points were identified, but further validation would be needed. From our data, the piece of information that shows the most promise of being generalizable with further model refining is short-term survival. The train-test data indicated that the gene most correlated to short survival given a four year threshold was CASP-3, with a regression coefficient of 0.457807. In addition, the gene expressed that led to the least amount of four-year survival was BCL2, which had a logistic regression coefficient of -0.429741.

One of the biggest ethical implementations of this work revolves around the ability for a model to have predictive abilities of survival regarding cancer. Although a model is a great way to make predictions, misclassification is possible (as shown by our model more times than ideal), which can lead to both false positive and false negative results, creating serious dilemmas. False positives or negatives can lead to over- or under-treatment, affecting patient care and potentially causing medical harm. Beyond clinical impact, inaccurate predictions may heighten patient anxiety or foster false hope. Ethically, such models must be used with caution, ensuring transparency about their limitations, fairness across patient groups, and accountability in how predictions are interpreted and applied.

Limitations and Future Work:

The biggest limitation from this project was our model's simplicity. Something as complex as breast cancer analysis requires a holistic approach, whereas we only focussed on four genes linked to cellular apoptosis. With something as complex as cancer, only looking at a few genes and survival outcomes is not enough to create a reliable and accurate model, as shown by our results. In addition, with our data set, only 75 samples fell under the BRCA classification, giving a much smaller sample size than ideal, therefore making it harder for our results to be generalized to broader contexts. Furthermore, our lack of proper external validation made our model centric on our 75 data points and the generated train-test data points, making our overall model less credible. Overall, the numerous shortcomings of our model and data set provide numerous limitations, but these can be addressed in future research.

Regarding future expansions of this project, looking at more genes involved with apoptosis resistance and breast cancer would be beneficial, as these would help to indicate if there are indeed better genes than the ones selected to predict short

versus long-term survival. In addition, looking at various other hallmarks of cancer in supplement to apoptosis evasion, would help to make the data analysis more holistic, indicating new relationships. Finally, using a larger, more clinical set of data for a similar project would help to provide greater external validation for the project, helping to explore specific time durations, tumor sizes, locations, etc. and how these supplement the genes and patient survival.

NOTES FROM YOUR TEAM:

In class on 10/23/25, we discussed our opinions on which cancer to focus on and whether we were interested in pursuing a single cancer focus or looking into multiple different cancer types. We also communicated how we would divide up the work and who should complete which parts of the notebook. Over the weekend, we decided on which hallmark we would like to focus on, and from there, we worked independently on our sections. Then, on 10/25/25, we met together, shared our work, and finished up the code together. Since then we ran into an issue and had to come up with a different question due to complications. When we finally came to an agreement on a new one we worked together to reframe our code to run. Throughout the remaining time, we developed our code to completion, refined our model, and then drew conclusions on our data.

QUESTIONS FOR YOUR TA:

None.